LAPORAN PRAKTIKUM MOBILE PROGRAMMING
MODUL 13 OPERASI CRUD
DI FLUTTER DENGAN REST API

Nama : Muhamad Kevin Dwi Arisandi

NIM : 240605110252

Kelas : Mobile Programming Praktikum A

Tanggal : 05 November 2026

JURUSAN TEKNIK INFORMATIKA

FAKULTAS SAINS DAN TEKNOLOGI

UNIVERSITAS ISLAM NEGERI MAULANA MALIK IBRAHIM

MALANG

GANJIL 2025/2026

I. Tujuan

Praktikum ini bertujuan memahami alur Create, Read, Update, dan Delete pada aplikasi Flutter yang berkomunikasi dengan REST API. Permintaan HTTP dikirim secara asinkron, respons JSON diubah menjadi model Post, sedangkan StatefulWidget dan setState() mengatur perubahan tampilan. Dialog, Card, menu aksi, dan SnackBar digunakan untuk menerima input serta memberi umpan balik.

II. Langkah Kerja

1. Membuat proyek flutter_crud_api dan menambahkan dependensi http, intl, serta google_fonts pada pubspec.yaml, lalu menjalankan flutter pub get.

2. Membuat model Post untuk memetakan JSON menjadi data id, userId, title, body, dan waktu yang ditampilkan. Menyiapkan HomePage sebagai StatefulWidget dengan variabel createdPost.

3. Menyusun halaman awal menggunakan Scaffold, AppBar, dan tombol tambah. Saat createdPost belum tersedia, layar menampilkan teks demo; ketika terisi, data disajikan dalam Card.

4. Membuat AlertDialog tambah berisi Nama dan Pekerjaan, validator kolom wajib, serta tombol Simpan yang memanggil createPost().

5. Mengirim POST ke endpoint /posts. Jika respons berhasil, data dibaca dari JSON dan dimasukkan ke createdPost melalui setState(), lalu hasil ditampilkan di Card.

6. Menambahkan PopupMenuButton untuk Update dan Delete. Dialog update menampilkan nilai lama, lalu updatePost() mengirim PUT dan mengubah Card. Dialog hapus meminta konfirmasi sebelum deletePost() mengirim DELETE dan mengosongkan state.

7. Menguji urutan tampilan awal, validasi, tambah, update, dan hapus pada perangkat Android serta mendokumentasikan pesan SnackBar yang tampak.

III. Penjelasan Kode Program Utama

Cuplikan berikut merangkum alur pada Modul 13. Screenshot membuktikan keadaan UI; berkas sumber lengkap aplikasi yang dijalankan tidak dilampirkan.

1. Create dengan POST

```
final response = await http.post(
  Uri.parse('https://jsonplaceholder.typicode.com/posts'),
  headers: {'Content-type': 'application/json; charset=UTF-8'},
  body: jsonEncode({'title': title, 'body': body, 'userId': 1}),
);
if (!mounted) return;
if (response.statusCode == 201) {
  final data = jsonDecode(response.body);
  setState(() => createdPost = Post.fromJson(data));
}
```

createPost() mengirim data Nama sebagai title dan Pekerjaan sebagai body. Respons 201 diparsing menjadi Post dan disimpan dalam createdPost, sehingga teks awal berganti menjadi Card. Input kosong ditolak melalui validasi dialog sebelum permintaan dikirim.

2. Update dengan PUT

```
final response = await http.put(
  Uri.parse('https://jsonplaceholder.typicode.com/posts/1'),
  headers: {'Content-type': 'application/json; charset=UTF-8'},
  body: jsonEncode({
    'id': 1, 'title': title, 'body': body, 'userId': 1,
  }),
);
if (!mounted) return;
if (response.statusCode == 200) {
  final data = jsonDecode(response.body);
  setState(() {
    createdPost = Post(
      id: data['id'], userId: data['userId'] ?? createdPost!.userId,
      title: data['title'], body: data['body'],
      createdAt: createdPost!.createdAt,
    );
  });
}
```

Contoh updatePost() pada modul memakai URL /posts/1 dan ID 1 di body. Setelah respons 200, createdPost diperbarui sambil mempertahankan waktu awal. Karena itu, ID pada screenshot berubah dari 101 saat Create menjadi 1 setelah Update; perubahan ini mengikuti kode demo.

3. Delete dan pembaruan tampilan

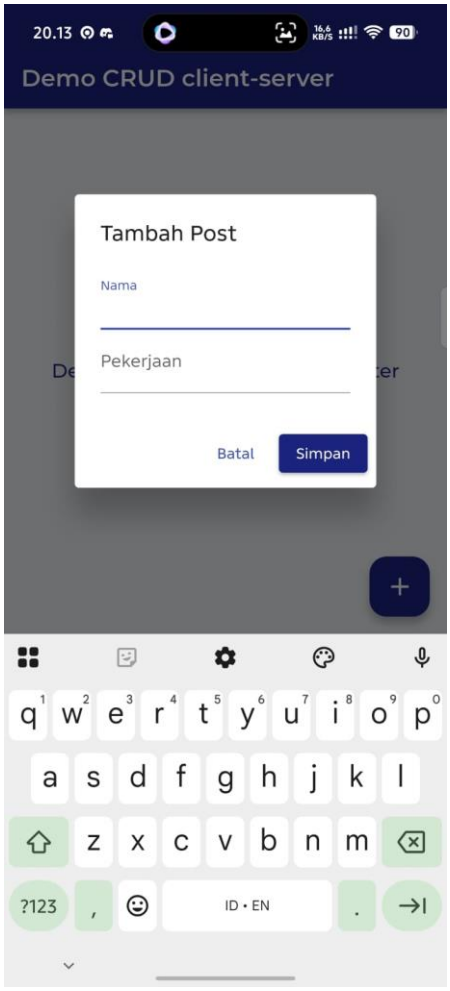
```
final response = await http.delete(
  Uri.parse('https://jsonplaceholder.typicode.com/posts/$id'),
  headers: {'Content-type': 'application/json; charset=UTF-8'},
);
if (!mounted) return;
if (response.statusCode == 200) {
  setState(() => createdPost = null);
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(content: Text('Post berhasil dihapus!')),
  );
}
```

deletePost() menghapus data yang dipilih dari tampilan setelah respons sukses. AlertDialog meminta konfirmasi, sedangkan SnackBar memberi pesan hasil. Permintaan gagal dan kesalahan jaringan ditangani dengan pesan error. Dalam demo JSONPlaceholder, perubahan pada Card merupakan state lokal dan respons simulasi API, bukan bukti penyimpanan permanen di server.

IV. Screenshot Hasil

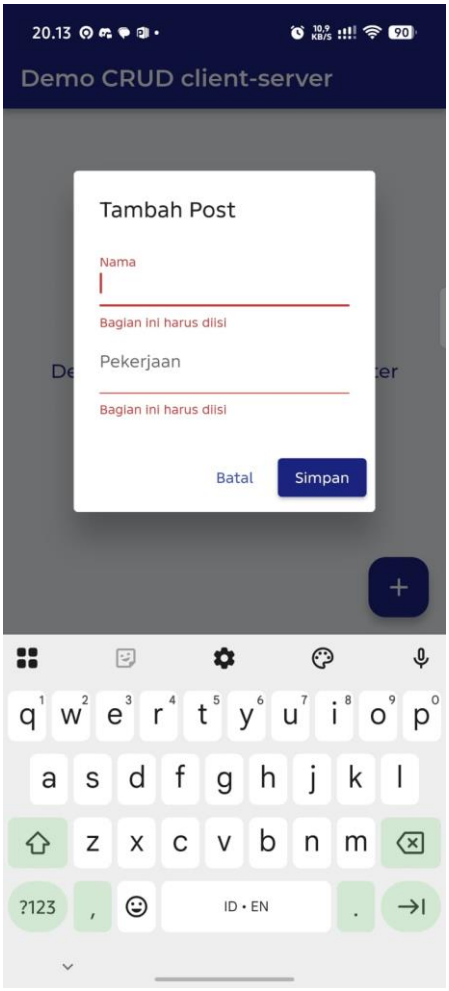


Gambar 1. Tampilan awal sebelum ada post.

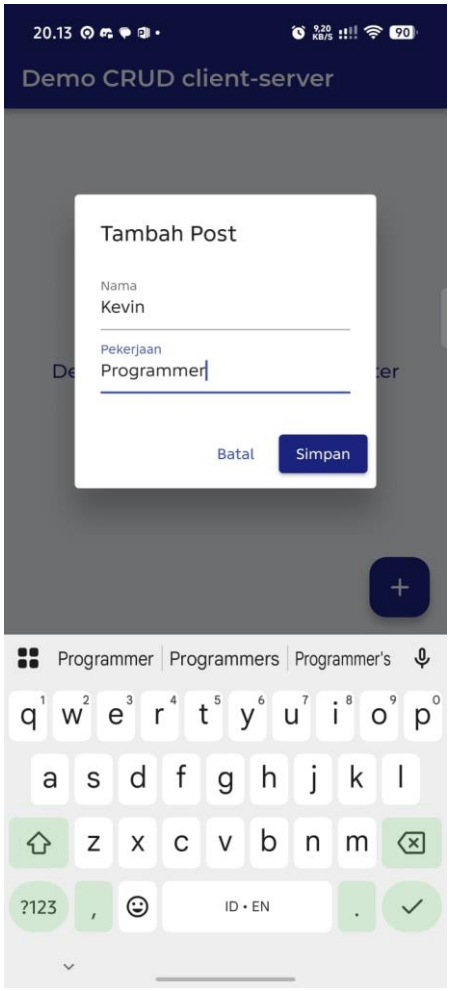


Gambar 2. Dialog Tambah Post terbuka.

Halaman awal menampilkan teks “Demo CRUD client-server di Flutter” dan tombol tambah. Tombol tersebut membuka dialog dengan kolom Nama dan Pekerjaan, serta aksi Batal dan Simpan.

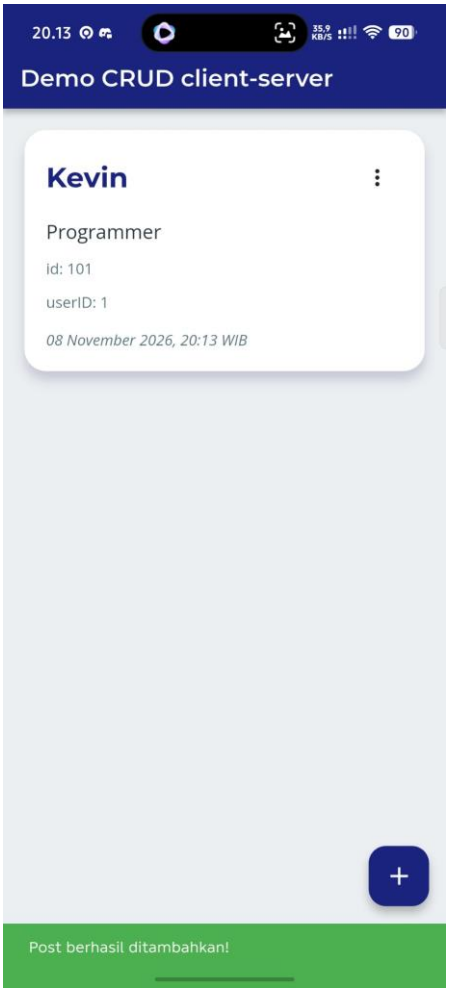


Gambar 3. Validasi kolom kosong.

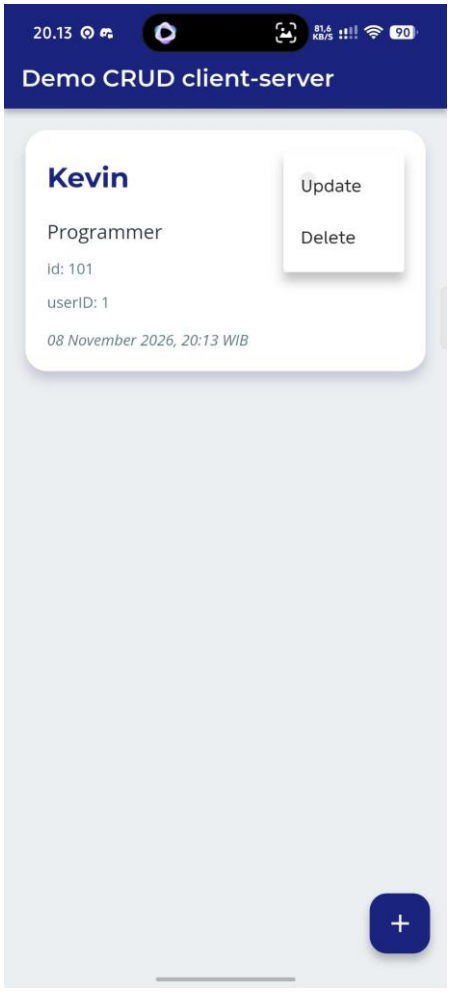


Gambar 4. Form diisi Kevin dan Programmer.

Ketika Simpan ditekan tanpa input, kedua kolom menampilkan “Bagian ini harus diisi”. Setelah diisi, data Kevin dan Programmer siap dikirim melalui aksi Simpan.

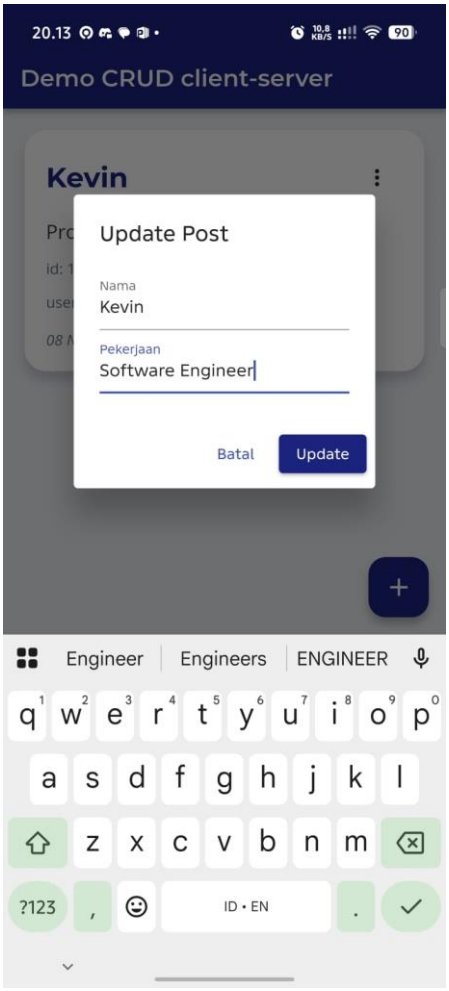


Gambar 5. Card hasil Create dan SnackBar sukses.

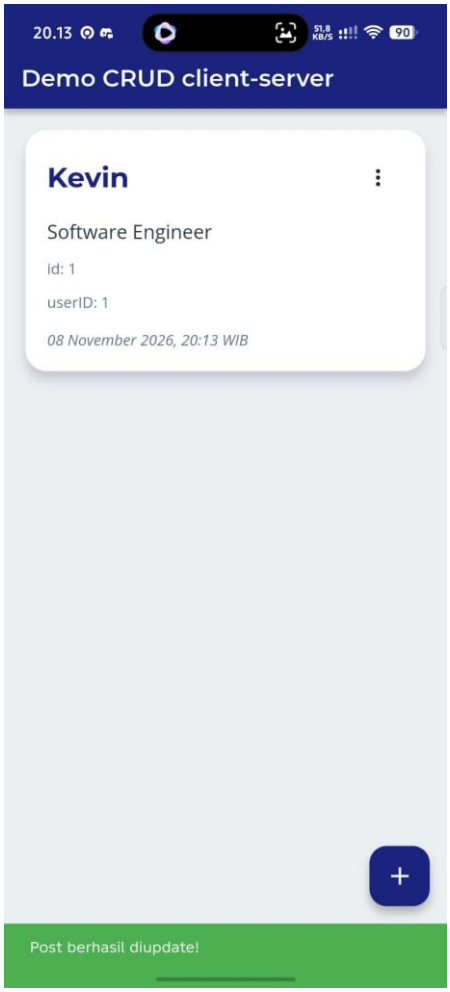


Gambar 6. Menu Update dan Delete.

Card menampilkan Kevin, Programmer, id 101, userID 1, dan waktu 08 November 2026 pukul 20:13 WIB. SnackBar hijau menyatakan post berhasil ditambahkan. Menu tiga titik menyediakan aksi Update dan Delete.



Gambar 7. Dialog mengubah pekerjaan.



Gambar 8. Card setelah Update dan SnackBar sukses.

Dialog update mempertahankan nama Kevin dan mengubah pekerjaan menjadi Software Engineer. Sesudahnya Card menampilkan pekerjaan baru dengan id 1, disertai pesan “Post berhasil diupdate!”.



Gambar 9. Konfirmasi penghapusan.



Gambar 10. Tampilan awal kembali setelah Delete.

Dialog meminta konfirmasi sebelum post dihapus. Setelah aksi Hapus, Card hilang, teks demo kembali muncul, dan SnackBar hijau menunjukkan “Post berhasil dihapus!”. Tidak ada screenshot kegagalan jaringan atau respons error.

V. Kesimpulan

POST, PUT, dan DELETE mengubah createdPost sesuai respons API. setState() menampilkan Card, memperbarui isinya, lalu mengembalikan layar awal setelah hapus. Validasi dan SnackBar memberi umpan balik. JSONPlaceholder menyediakan respons simulasi; screenshot tidak membuktikan penyimpanan permanen di server.